

目录

图论易错点与建模提示	2
先考虑的等价变换	2
DFS 边分类	2
传递闭包	2
最短路	2
匹配与覆盖	3
Tarjan 连通性算法	3
有向图强连通分量	3
桥、边双连通分量与桥树	4
点双连通分量与圆方树	6
Dinic 最大流	8
最小费用最大流	10
网络流建模技巧	13
最大权闭合子图	13
最大密度子图	14
上下界可行流	15
有源汇上下界最大流 / 最小流	16
其他常见建模	18
图匹配	18
Hopcroft-Karp 二分图最大匹配	18
偏序集最大反链：拆点、匹配与构造	19
怎样实际取出反链	19
例子：整除偏序	20
方法二：原笔记的“链尾向链首跳”	22
二分图最大匹配的可行边与必须边	22
二分图最大权完美匹配	25
一般图最大匹配 (Edmonds Blossom)	27
三元环与四元环计数	29
图论常用技巧	30
最短路势能	30
最短路 DAG	30
MST 换边	30
补图 BFS	31
函数图	31
欧拉路检查表	31
离线删除	31
Kruskal 重构树	31
Euler 序 RMQ 求 LCA	34
Hierholzer 欧拉路	36
2-SAT	38
Capacity scaling 最小费用循环流	40
模板	40

从循环流得到最小费用最大流	44
有负费用边、负圈时的上下界做法	44

图论易错点与建模提示

先考虑的等价变换

- 连边条件难描述时，检查反图、补图或把“不能同时选”改成冲突边。
- 动态删边可离线倒序变成加边；连通性通常交给并查集。
- 点权路径问题可拆点；边权限制有时可放到 Kruskal 重构树上转成 LCA。
- DAG 上的可达偏序问题常先做传递闭包，再转成二分图匹配。
- 更多补图 BFS、势能、MST 换边、函数图和欧拉图技巧见 08-techniques.md。
- Euler 序 RMQ 求 LCA 的完整代码见 10-euler-tour-lca.md；2-SAT 完整代码见 12-two-sat.md。

DFS 边分类

无向图 DFS 必须用边编号跳过父边，否则重边会被误判。对一条非树边，它的两个端点在 DFS 树上具有祖先关系。

有向图可按访问状态分类：

- 指向未访问点的是树边；
- 指向当前递归栈中点的是返祖边；
- 指向已退出点时，再用进入/退出时间判断前向边或横叉边。

“看到已访问点就一定指向祖先”只对无向 DFS 的非树边成立，不能直接套到有向图。

传递闭包

顶点数中等时，bitset 版 Floyd 很实用。若 `reachable[i][k]` 为真，就把第 k 行并入第 i 行；复杂度约为 $O(n^3/w)$ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

template<size_t MaximumSize>
void transitive_closure(vector<bitset<MaximumSize>>& reachable) {
    int size = (int)reachable.size();
    assert(size <= (int)MaximumSize);
    for (int vertex = 0; vertex < size; ++vertex) {
        reachable[vertex].set(vertex);
    }
    for (int middle = 0; middle < size; ++middle) {
        for (int from = 0; from < size; ++from) {
            if (reachable[from][middle]) {
                reachable[from] |= reachable[middle];
            }
        }
    }
}
```

最短路

- Johnson 重标号先增加虚拟源并用 Bellman-Ford 求势能 h 。新边权为 $w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$ ；Dijkstra 后要还原距离。

- 若只关心满足同余条件的最小可达值，可把余数当顶点做同余最短路。
- 最短路上的边满足 $\text{distance}[u] + \text{weight} == \text{distance}[v]$ ；筛出后通常得到最短路 DAG。

匹配与覆盖

- Berge 引理：匹配最大当且仅当不存在增广路。
- Hall 定理：二分图存在覆盖左部的匹配，当且仅当每个左部子集 S 都满足 $|N(S)| \geq |S|$ 。
- 柯尼希定理：二分图最大匹配数等于最小点覆盖数；因此最大独立集大小为 $|V| - |M|$ 。
- DAG 的顶点不交最小路径覆盖等于 $|V|$ 减去拆点二分图的最大匹配数。
- 若允许路径重复经过顶点，应先用可达关系建边，而不是只使用原图边。

二分图最大匹配中可行边、必须边的一般判定，以及偏序集最大反链的实际构造见 06-match.md。非完美匹配还要处理从自由左点出发和走向自由右点的交错路，不能只照搬完美匹配的 SCC 判据。

Tarjan 连通性算法

无向图必须用边编号跳过 DFS 树中的父边，不能用“跳过父亲节点”代替，否则重边会被误判。下列模板均使用 0-based 点编号，并允许图不连通。

有向图强连通分量

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class StronglyConnectedComponents {
    int node_count;
    int timer = 0;
    vector<vector<int>>> graph;
    vector<int> dfn;
    vector<int> low;
    vector<int> stack;
    vector<bool> in_stack;

    void dfs(int node) {
        dfn[node] = low[node] = ++timer;
        stack.push_back(node);
        in_stack[node] = true;
        for (int next : graph[node]) {
            if (dfn[next] == 0) {
                dfs(next);
                low[node] = min(low[node], low[next]);
            } else if (in_stack[next]) {
                low[node] = min(low[node], dfn[next]);
            }
        }
        if (low[node] != dfn[node]) return;

        components.push_back({});
        while (true) {
            int current = stack.back();
            stack.pop_back();
            components.back().push_back(current);
            in_stack[current] = false;
        }
    }

    vector<vector<int>> components;

    void run() {
        node_count = graph.size();
        dfn.resize(node_count);
        low.resize(node_count);
        stack.resize(node_count);
        in_stack.resize(node_count);
        components.clear();
        for (int i = 0; i < node_count; i++) {
            if (dfn[i] == 0) dfs(i);
        }
    }
};
```

```

        in_stack[current] = false;
        component_of[current] = (int)components.size() - 1;
        components.back().push_back(current);
        if (current == node) break;
    }
}

public:
    vector<int> component_of;
    vector<vector<int>> components;

    explicit StronglyConnectedComponents(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count),
          dfn(initial_node_count), low(initial_node_count),
          in_stack(initial_node_count), component_of(initial_node_count, -1) {}

    void add_edge(int from, int to) {
        graph[from].push_back(to);
    }

    int build() {
        for (int node = 0; node < node_count; ++node) {
            if (dfn[node] == 0) dfs(node);
        }
        return (int)components.size();
    }
};

```

同一个强连通分量中的点可以互相到达。缩点时遍历所有原边，仅在两端分量编号不同的情况下加边；需要简单 DAG 时再排序去重。复杂度为 $O(V + E)$ 。

桥、边双连通分量与桥树

```

class EdgeBiconnectedComponents {
    struct AdjacentEdge {
        int to;
        int edge_id;
    };

    int node_count;
    int timer = 0;
    vector<vector<AdjacentEdge>> graph;
    vector<pair<int, int>> edges;
    vector<int> dfn;
    vector<int> low;

    void find_bridges(int node, int parent_edge) {
        dfn[node] = low[node] = ++timer;
        for (const AdjacentEdge& edge : graph[node]) {
            if (edge.edge_id == parent_edge) continue;
            if (dfn[edge.to] == 0) {
                find_bridges(edge.to, edge.edge_id);
                low[node] = min(low[node], low[edge.to]);
            }
        }
    }
};

```

```

        if (low[edge.to] > dfn[node]) is_bridge[edge.edge_id] = true;
    } else {
        low[node] = min(low[node], dfn[edge.to]);
    }
}
}

void paint_component(int start, int component_id) {
    stack<int> pending;
    pending.push(start);
    component_of[start] = component_id;
    while (!pending.empty()) {
        int node = pending.top();
        pending.pop();
        for (const AdjacentEdge& edge : graph[node]) {
            if (is_bridge[edge.edge_id] || component_of[edge.to] != -1) {
                continue;
            }
            component_of[edge.to] = component_id;
            pending.push(edge.to);
        }
    }
}

public:
    vector<bool> is_bridge;
    vector<int> component_of;

    explicit EdgeBiconnectedComponents(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count),
          dfn(initial_node_count), low(initial_node_count),
          component_of(initial_node_count, -1) {}

    int add_edge(int lhs, int rhs) {
        int edge_id = (int)edges.size();
        edges.push_back({lhs, rhs});
        graph[lhs].push_back({rhs, edge_id});
        graph[rhs].push_back({lhs, edge_id});
        return edge_id;
    }

    int build() {
        is_bridge.assign(edges.size(), false);
        for (int node = 0; node < node_count; ++node) {
            if (dfn[node] == 0) find_bridges(node, -1);
        }
        int component_count = 0;
        for (int node = 0; node < node_count; ++node) {
            if (component_of[node] == -1) {
                paint_component(node, component_count++);
            }
        }
        return component_count;
    }

```

```

    }

    vector<vector<int>> bridge_forest(int component_count) const {
        vector<vector<int>> forest(component_count);
        for (int edge_id = 0; edge_id < (int)edges.size(); ++edge_id) {
            if (!is_bridge[edge_id]) continue;
            auto [lhs, rhs] = edges[edge_id];
            int lhs_component = component_of[lhs];
            int rhs_component = component_of[rhs];
            forest[lhs_component].push_back(rhs_component);
            forest[rhs_component].push_back(lhs_component);
        }
        return forest;
    }
};

```

删除所有桥后，每个连通块就是一个边双连通分量；把每个分量缩成点、每座桥保留成边，得到的是森林。复杂度为 $O(V + E)$ 。

点双连通分量与圆方树

一个孤立点也被视为一个点双连通分量。圆方树中前 `components.size()` 个节点代表点双分量，之后的节点代表割点。

```

class VertexBiconnectedComponents {
    struct AdjacentEdge {
        int to;
        int edge_id;
    };

    int node_count;
    int timer = 0;
    int edge_count = 0;
    vector<vector<AdjacentEdge>> graph;
    vector<int> dfn;
    vector<int> low;
    vector<int> vertex_stack;

    void dfs(int node, int parent_edge) {
        dfn[node] = low[node] = ++timer;
        int child_count = 0;
        for (const AdjacentEdge& edge : graph[node]) {
            if (edge.edge_id == parent_edge) continue;
            if (dfn[edge.to] == 0) {
                ++child_count;
                vertex_stack.push_back(edge.to);
                dfs(edge.to, edge.edge_id);
                low[node] = min(low[node], low[edge.to]);
                if (low[edge.to] >= dfn[node]) {
                    if (parent_edge != -1 || child_count > 1) {
                        is_cut_vertex[node] = true;
                    }
                    components.push_back({node});
                    while (components.back().back() != edge.to) {

```

```

        components.back().push_back(vertex_stack.back());
        vertex_stack.pop_back();
    }
} else {
    low[node] = min(low[node], dfn[edge.to]);
}
}
if (parent_edge == -1 && child_count == 0) {
    components.push_back({node});
}
}

public:
vector<bool> is_cut_vertex;
vector<vector<int>> components;

explicit VertexBiconnectedComponents(int initial_node_count)
: node_count(initial_node_count), graph(initial_node_count),
  dfn(initial_node_count), low(initial_node_count),
  is_cut_vertex(initial_node_count) {}

void add_edge(int lhs, int rhs) {
    int edge_id = edge_count++;
    graph[lhs].push_back({rhs, edge_id});
    graph[rhs].push_back({lhs, edge_id});
}

int build() {
    for (int node = 0; node < node_count; ++node) {
        if (dfn[node] != 0) continue;
        vertex_stack.push_back(node);
        dfs(node, -1);
        vertex_stack.clear();
    }
    return (int)components.size();
}

vector<vector<int>> block_cut_forest() const {
    int component_count = (int)components.size();
    vector<int> cut_id(node_count, -1);
    int tree_size = component_count;
    for (int node = 0; node < node_count; ++node) {
        if (is_cut_vertex[node]) cut_id[node] = tree_size++;
    }

    vector<vector<int>> forest(tree_size);
    for (int component_id = 0;
        component_id < component_count; ++component_id) {
        for (int node : components[component_id]) {
            if (!is_cut_vertex[node]) continue;
            forest[component_id].push_back(cut_id[node]);
            forest[cut_id[node]].push_back(component_id);
        }
    }
}

```

```

    }
}
return forest;
}
};

```

复杂度为 $O(V + E)$ 。代码使用递归 DFS；若图可能是一条百万级长链，应改成显式栈或确认评测环境栈空间足够。

Dinic 最大流

下列实现是完整、可独立使用的 Dinic。调用 `add_edge(from, to, capacity)` 时会自动加入容量为零的反向边，不要手动补反边。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class Dinic {
    using i64 = long long;

public:
    struct EdgeHandle {
        int from;
        int index;
    };

private:
    struct Edge {
        int to;
        int reverse_id;
        i64 capacity;
    };

    int node_count;
    vector<vector<Edge>> graph;
    vector<int> level;
    vector<int> current_edge;

    bool build_level_graph(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> que;
        level[source] = 0;
        que.push(source);
        while (!que.empty()) {
            int node = que.front();
            que.pop();
            for (const Edge& edge : graph[node]) {
                if (edge.capacity > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[node] + 1;
                    que.push(edge.to);
                }
            }
        }
    }
};

```



```

    }
    }
    }
    return level[sink] != -1;
}

i64 send_flow(int node, int sink, i64 available) {
    if (node == sink) return available;
    for (int& edge_id = current_edge[node];
         edge_id < (int)graph[node].size(); ++edge_id) {
        Edge& edge = graph[node][edge_id];
        if (edge.capacity == 0 || level[edge.to] != level[node] + 1) {
            continue;
        }
        i64 pushed = send_flow(edge.to, sink, min(available, edge.capacity));
        if (pushed == 0) continue;
        edge.capacity -= pushed;
        graph[edge.to][edge.reverse_id].capacity += pushed;
        return pushed;
    }
    return 0;
}

public:
    explicit Dinic(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count),
          level(initial_node_count), current_edge(initial_node_count) {}

    EdgeHandle add_edge(int from, int to, i64 capacity) {
        assert(0 <= from && from < node_count);
        assert(0 <= to && to < node_count);
        assert(capacity >= 0);
        int from_id = (int)graph[from].size();
        int to_id = (int)graph[to].size() + (from == to);
        graph[from].push_back({to, to_id, capacity});
        graph[to].push_back({from, from_id, 0});
        return {from, from_id};
    }

    i64 flow_on(EdgeHandle handle) const {
        const Edge& edge = graph[handle.from][handle.index];
        return graph[edge.to][edge.reverse_id].capacity;
    }

    void disable_edge(EdgeHandle handle) {
        int to = graph[handle.from][handle.index].to;
        int reverse_id = graph[handle.from][handle.index].reverse_id;
        graph[handle.from][handle.index].capacity = 0;
        graph[to][reverse_id].capacity = 0;
    }

    i64 max_flow(int source, int sink,
                 i64 flow_limit = numeric_limits<i64>::max()) {

```

```

    assert(source != sink);
    i64 total_flow = 0;
    while (total_flow < flow_limit && build_level_graph(source, sink)) {
        fill(current_edge.begin(), current_edge.end(), 0);
        while (total_flow < flow_limit) {
            i64 pushed = send_flow(source, sink, flow_limit - total_flow);
            if (pushed == 0) break;
            total_flow += pushed;
        }
    }
    return total_flow;
}

vector<bool> min_cut_side(int source) const {
    vector<bool> reachable(node_count);
    queue<int> que;
    reachable[source] = true;
    que.push(source);
    while (!que.empty()) {
        int node = que.front();
        que.pop();
        for (const Edge& edge : graph[node]) {
            if (edge.capacity > 0 && !reachable[edge.to]) {
                reachable[edge.to] = true;
                que.push(edge.to);
            }
        }
    }
    return reachable;
}
};

```

`add_edge` 返回的句柄始终指向这条正向边；`flow_on(handle)` 读取当前流量，`disable_edge(handle)` 同时清空正反残量容量，主要供上下界流删除辅助边。普通最大流不需要保存句柄。

`min_cut_side(source)` 应在 `max_flow` 结束后调用，返回残量网络中源点仍能到达的点，也就是最小割的源侧。一般网络上常用复杂度上界为 $O(V^2E)$ ；单位容量网络和二分图网络会更快。递归深度可能达到 V ，若题目给出极深链且栈空间很小，需要改成迭代 DFS 或增大栈空间。

最小费用最大流

实现使用势能加 Dijkstra。第一次用 SPFA 计算合法初始势能，因此允许负费用边；残量网络中不得存在从源点可达的负费用环。每次调用 `add_edge` 会自动加入容量为零、费用取反的反向边。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class MinCostMaxFlow {
    using i64 = long long;
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;

    struct Edge {

```

```

    int to;
    int reverse_id;
    i64 capacity;
    i64 cost;
};

int node_count;
vector<vector<Edge>> graph;

vector<i64> initial_potential(int source) const {
    vector<i64> distance(node_count, inf);
    vector<bool> in_queue(node_count, false);
    queue<int> que;
    distance[source] = 0;
    in_queue[source] = true;
    que.push(source);
    while (!que.empty()) {
        int node = que.front();
        que.pop();
        in_queue[node] = false;
        for (const Edge& edge : graph[node]) {
            if (edge.capacity == 0 || distance[node] == inf) continue;
            if (distance[edge.to] > distance[node] + edge.cost) {
                distance[edge.to] = distance[node] + edge.cost;
                if (!in_queue[edge.to]) {
                    in_queue[edge.to] = true;
                    que.push(edge.to);
                }
            }
        }
    }
    for (i64& value : distance) {
        if (value == inf) value = 0;
    }
    return distance;
}

public:
    explicit MinCostMaxFlow(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count) {}

    void add_edge(int from, int to, i64 capacity, i64 cost) {
        assert(0 <= from && from < node_count);
        assert(0 <= to && to < node_count);
        assert(capacity >= 0);
        int from_id = (int)graph[from].size();
        int to_id = (int)graph[to].size();
        graph[from].push_back({to, to_id, capacity, cost});
        graph[to].push_back({from, from_id, 0, -cost});
    }

    pair<i64, i64> min_cost_max_flow(
        int source, int sink,

```

```

i64 flow_limit = numeric_limits<i64>::max();
vector<i64> potential = initial_potential(source);
vector<i64> distance(node_count);
vector<int> previous_node(node_count);
vector<int> previous_edge(node_count);
i64 total_flow = 0;
i64 total_cost = 0;

while (total_flow < flow_limit) {
    fill(distance.begin(), distance.end(), inf);
    priority_queue<pair<i64, int>,
                  vector<pair<i64, int>>,
                  greater<pair<i64, int>>> > que;
    distance[source] = 0;
    que.push({0, source});
    while (!que.empty()) {
        auto [current_distance, node] = que.top();
        que.pop();
        if (current_distance != distance[node]) continue;
        for (int edge_id = 0;
              edge_id < (int)graph[node].size(); ++edge_id) {
            const Edge& edge = graph[node][edge_id];
            if (edge.capacity == 0) continue;
            i64 reduced_cost =
                edge.cost + potential[node] - potential[edge.to];
            i64 next_distance = current_distance + reduced_cost;
            if (next_distance < distance[edge.to]) {
                distance[edge.to] = next_distance;
                previous_node[edge.to] = node;
                previous_edge[edge.to] = edge_id;
                que.push({next_distance, edge.to});
            }
        }
    }
    if (distance[sink] == inf) break;

    for (int node = 0; node < node_count; ++node) {
        if (distance[node] != inf) potential[node] += distance[node];
    }

    i64 pushed = flow_limit - total_flow;
    for (int node = sink; node != source; node = previous_node[node]) {
        const Edge& edge = graph[previous_node[node]][previous_edge[node]];
        pushed = min(pushed, edge.capacity);
    }
    for (int node = sink; node != source; node = previous_node[node]) {
        Edge& edge = graph[previous_node[node]][previous_edge[node]];
        edge.capacity -= pushed;
        graph[node][edge.reverse_id].capacity += pushed;
    }
    total_flow += pushed;
    total_cost += pushed * potential[sink];
}

```

```

        return {total_flow, total_cost};
    }
};

```

设最终增广次数为 F ，复杂度约为 $O(VE + FE \log V)$ ；第一项来自初始 SPFA 的常见估计，SPFA 的理论最坏复杂度为 $O(VE)$ 。容量乘费用以及总费用必须能装入 `long long`。若只需要恰好发送 k 单位流，传入 `flow_limit=k` 并检查返回流量是否等于 k 。

网络流建模技巧

本文件代码直接使用 03-maxflow.md 中的 Dinic。

```
using flow_i128 = __int128_t;
```

最大权闭合子图

有向图中选择点集 S ，并要求 $u \in S$ 且存在依赖边 $u \rightarrow v$ 时必有 $v \in S$ 。点有正负权，目标是最大化点权和。

- 权值 $w_i > 0$: 连 `source` $\rightarrow$ `i`，容量 w_i ；
- 权值 $w_i < 0$: 连 `i` $\rightarrow$ `sink`，容量 $-w_i$ ；
- 依赖 $u \rightarrow v$: 连容量 `infinity` 的 `u` $\rightarrow$ `v`。

答案是“所有正权之和减最小割”。最大流结束后残量网络中源点可达的原图点就是一组最优解。

```

struct ClosureResult {
    long long value;
    vector<int> selected;
};

ClosureResult maximum_weight_closure(
    const vector<long long>& weight,
    const vector<pair<int, int>>& dependencies) {
    int n = (int)weight.size();
    int source = n, sink = n + 1;
    flow_i128 positive_sum = 0;
    for (long long value : weight) {
        if (value > 0) positive_sum += value;
        assert(value != numeric_limits<long long>::min());
    }
    assert(positive_sum < numeric_limits<long long>::max());
    long long infinity = (long long)positive_sum + 1;

    Dinic flow(n + 2);
    for (int i = 0; i < n; ++i) {
        if (weight[i] > 0) flow.add_edge(source, i, weight[i]);
        if (weight[i] < 0) flow.add_edge(i, sink, -weight[i]);
    }
    for (auto [u, v] : dependencies) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        flow.add_edge(u, v, infinity);
    }
}

```

```

long long cut = flow.max_flow(source, sink);
vector<bool> source_side = flow.min_cut_side(source);
vector<int> selected;
for (int i = 0; i < n; ++i) {
    if (source_side[i]) selected.push_back(i);
}
return {(long long)positive_sum - cut, selected};
}

```

`infinity = positive_sum + 1` 已经足够：空集给出容量至多 `positive_sum` 的有限割，所以最小割绝不会切断依赖边。代码要求正权总和能放入 `long long`；否则最大流容量本身也需要换成 `__int128`。

最大密度子图

对无自环无向图选择非空点集 S ，密度定义为诱导子图边数除以点数：

$$\rho(S) = \frac{|E(S)|}{|S|}.$$

判断是否存在 $\rho(S) > g$ 时，为每个点建立网络：

- `source` -> `v` 容量 m ;
- `v` -> `sink` 容量 $m + 2g - \deg(v)$;
- 每条无向边 (u, v) 加容量为 1 的 `u` -> `v` 和 `v` -> `u`。

若割的源侧原图点集为 S ，割容量恰为

$$nm + 2g|S| - 2|E(S)|.$$

因此最小割小于 nm 当且仅当存在密度大于 g 的非空子图。下面把所有容量乘 `scale` 后做整数二分，避免浮点容量误差；返回值绝对误差不超过 $1/(2 \text{ scale})$ 。

```

long double maximum_density_subgraph(
    int n, const vector<pair<int, int>>& edges,
    long long scale = 1000000) {
    assert(n > 0 && scale > 0);
    int m = (int)edges.size();
    vector<int> degree(n);
    for (auto [u, v] : edges) {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        ++degree[u];
        ++degree[v];
    }

    flow_i128 capacity_bound = (flow_i128)3 * m * scale;
    flow_i128 total_bound = (flow_i128)n * m * scale;
    assert(capacity_bound <= numeric_limits<long long>::max());
    assert(total_bound <= numeric_limits<long long>::max());
    long long base_capacity = (long long)((flow_i128)m * scale);
    long long baseline = (long long)total_bound;

    auto feasible = [&](long long density_scaled) {
        int source = n, sink = n + 1;
        Dinic flow(n + 2);

```

```

    for (int v = 0; v < n; ++v) {
        flow.add_edge(source, v, base_capacity);
        long long sink_capacity =
            (long long)(m - degree[v]) * scale + 2 * density_scaled;
        flow.add_edge(v, sink, sink_capacity);
    }
    for (auto [u, v] : edges) {
        flow.add_edge(u, v, scale);
        flow.add_edge(v, u, scale);
    }
    return flow.max_flow(source, sink, baseline) < baseline;
};

long long low = -1;
long long high = (long long)m * scale;
while (high - low > 1) {
    long long middle = low + (high - low) / 2;
    if (feasible(middle)) low = middle;
    else high = middle;
}
if (m == 0) return 0;
return (low + high) / (2.0L * scale);
}

```

一次判定是一遍最大流，总复杂度再乘 $O(\log(m \text{ scale}))$ 。平行边可以保留并分别计数；自环应预先按题意处理。若题目要求输出点集，在最终可行的 `low` 上再跑一次流并取 `min_cut_side(source)`。

上下界可行流

对边 $u \rightarrow v$ 的流量限制 $lower \leq f \leq upper$ ，先让它流过 `lower`，残量边容量变成 `upper-lower`。令 `balance[v]` 为下界造成的“流入减流出”：

- `balance[v] > 0`：加 `super_source -> v`；
- `balance[v] < 0`：加 `v -> super_sink`。

超级源的边全部满流，当且仅当存在可行环流。

```

struct BoundedEdge {
    int from, to;
    long long lower, upper;
};

bool feasible_circulation(int n, const vector<BoundedEdge>& edges) {
    int super_source = n, super_sink = n + 1;
    Dinic flow(n + 2);
    vector<flow_i128> balance(n);
    for (const BoundedEdge& edge : edges) {
        assert(0 <= edge.from && edge.from < n);
        assert(0 <= edge.to && edge.to < n);
        assert(0 <= edge.lower && edge.lower <= edge.upper);
        flow.add_edge(edge.from, edge.to, edge.upper - edge.lower);
        balance[edge.from] -= edge.lower;
        balance[edge.to] += edge.lower;
    }
}

```

```

flow_i128 demand = 0;
for (int v = 0; v < n; ++v) {
    if (balance[v] > 0) {
        assert(balance[v] <= numeric_limits<long long>::max());
        flow.add_edge(super_source, v, (long long)balance[v]);
        demand += balance[v];
    } else if (balance[v] < 0) {
        assert(-balance[v] <= numeric_limits<long long>::max());
        flow.add_edge(v, super_sink, (long long)-balance[v]);
    }
}
assert(demand <= numeric_limits<long long>::max());
return flow.max_flow(super_source, super_sink, (long long)demand) == demand;
}

```

有源汇上下界最大流 / 最小流

约定流值非负。先加辅助边 $\text{sink} \rightarrow \text{source}$ ，求出一组可行环流；辅助边上的流量就是当前 $s \rightarrow t$ 流值。随后必须在同一张残量网络里同时删除辅助边的正反残量边以及超级源汇的边：

- 最大流再跑 $\text{source} \rightarrow \text{sink}$;
- 最小流再跑 $\text{sink} \rightarrow \text{source}$ ，但最多撤回当前流值，避免把流值减成负数。

`Dinic::flow_on(handle)` 读取一条原始正向边当前已经流过的量。返回结果同时恢复每条输入边的实际流量。代码要求所有边容量总和能放进 `long long`；这也是当前 Dinic 能安全保存总流量的条件。

```

struct BoundedFlowResult {
    long long value;
    vector<long long> edge_flow;
};

optional<BoundedFlowResult> bounded_st_flow(
    int n, int source, int sink,
    const vector<BoundedEdge>& edges, bool maximize) {
    assert(0 <= source && source < n);
    assert(0 <= sink && sink < n && source != sink);
    int super_source = n, super_sink = n + 1;
    Dinic flow(n + 2);
    vector<flow_i128> balance(n);
    vector<Dinic::EdgeHandle> original_edges;
    flow_i128 capacity_sum = 0;

    for (const BoundedEdge& edge : edges) {
        assert(0 <= edge.from && edge.from < n);
        assert(0 <= edge.to && edge.to < n);
        assert(0 <= edge.lower && edge.lower <= edge.upper);
        original_edges.push_back(
            flow.add_edge(edge.from, edge.to, edge.upper - edge.lower));
        balance[edge.from] -= edge.lower;
        balance[edge.to] += edge.lower;
        capacity_sum += edge.upper;
    }
    assert(capacity_sum <= numeric_limits<long long>::max());
    long long infinity = (long long)max<flow_i128>(1, capacity_sum);
    auto auxiliary = flow.add_edge(sink, source, infinity);
}

```



```

vector<Dinic::EdgeHandle> super_edges;
flow_i128 demand = 0;
for (int v = 0; v < n; ++v) {
    if (balance[v] > 0) {
        assert(balance[v] <= numeric_limits<long long>::max());
        super_edges.push_back(flow.add_edge(
            super_source, v, (long long)balance[v]));
        demand += balance[v];
    } else if (balance[v] < 0) {
        assert(-balance[v] <= numeric_limits<long long>::max());
        super_edges.push_back(flow.add_edge(
            v, super_sink, (long long)-balance[v]));
    }
}
assert(demand <= numeric_limits<long long>::max());
if (flow.max_flow(super_source, super_sink, (long long)demand)
    != demand) {
    return nullopt;
}

long long base_value = flow.flow_on(auxiliary);
for (auto handle : super_edges) flow.disable_edge(handle);
flow.disable_edge(auxiliary);

flow_i128 answer = base_value;
if (maximize) {
    answer += flow.max_flow(source, sink);
} else {
    answer -= flow.max_flow(sink, source, base_value);
}
assert(0 <= answer && answer <= numeric_limits<long long>::max());

vector<long long> edge_flow(edges.size());
for (int i = 0; i < (int)edges.size(); ++i) {
    edge_flow[i] = edges[i].lower + flow.flow_on(original_edges[i]);
}
return BoundedFlowResult{(long long)answer, move(edge_flow)};
}

optional<BoundedFlowResult> maximum_bounded_flow(
    int n, int source, int sink, const vector<BoundedEdge>& edges) {
    return bounded_st_flow(n, source, sink, edges, true);
}

optional<BoundedFlowResult> minimum_bounded_flow(
    int n, int source, int sink, const vector<BoundedEdge>& edges) {
    return bounded_st_flow(n, source, sink, edges, false);
}

```

注意：只判断可行时仍可直接使用上一节的 `feasible_circulation`。最大值和最小值必须分别从初始网络求解，不能在已经求过最大流的残量网络上继续求最小流。

其他常见建模

- 最大流值已知时，可以给最大流加上恰好该流量的限制，再求最小费用，从而得到最小费用最大流。
 - 点权路径或点容量限制使用拆点： $\text{in}(v) \rightarrow \text{out}(v)$ 承载点的容量或费用。
 - 不要凭图形猜最小割：固定源侧点集，逐类列出真正从源侧指向汇侧的边，通常更不容易漏项。
-

图匹配

Hopcroft-Karp 二分图最大匹配

左右两侧分别使用 0-based 编号。只从左侧调用 `add_edge(left, right)`。复杂度为 $O(E\sqrt{V})$ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class HopcroftKarp {
    int left_count;
    int right_count;
    vector<vector<int>>> graph;
    vector<int> distance;

    bool bfs() {
        queue<int> que;
        fill(distance.begin(), distance.end(), -1);
        for (int left = 0; left < left_count; ++left) {
            if (match_left[left] == -1) {
                distance[left] = 0;
                que.push(left);
            }
        }
        bool found = false;
        while (!que.empty()) {
            int left = que.front();
            que.pop();
            for (int right : graph[left]) {
                int next_left = match_right[right];
                if (next_left == -1) {
                    found = true;
                } else if (distance[next_left] == -1) {
                    distance[next_left] = distance[left] + 1;
                    que.push(next_left);
                }
            }
        }
        return found;
    }

    bool dfs(int left) {
        for (int right : graph[left]) {
            int next_left = match_right[right];
            if (next_left == -1 ||
                (distance[next_left] == distance[left] + 1 && dfs(next_left))) {

```

```

        match_left[left] = right;
        match_right[right] = left;
        return true;
    }
}
distance[left] = -1;
return false;
}

public:
    vector<int> match_left;
    vector<int> match_right;

    HopcroftKarp(int left_size, int right_size)
        : left_count(left_size), right_count(right_size), graph(left_size),
          distance(left_size), match_left(left_size, -1),
          match_right(right_size, -1) {}

    void add_edge(int left, int right) {
        graph[left].push_back(right);
    }

    int max_matching() {
        int matching_size = 0;
        while (bfs()) {
            for (int left = 0; left < left_count; ++left) {
                if (match_left[left] == -1 && dfs(left)) ++matching_size;
            }
        }
        return matching_size;
    }
};

```

偏序集最大反链：拆点、匹配与构造

Dilworth 定理：有限偏序集的最大反链大小等于最小链划分大小。把每个元素 i 拆成左点 i_L 和右点 i_R ；对每个严格偏序关系 $i < j$ 连边 $i_L \rightarrow j_R$ 。若最大匹配大小为 m ，则最大反链大小为 $n - m$ 。

左右副本是二分图中互不相同的点，所以同一个元素的左右副本可以同时参与两条匹配边。例如匹配中同时出现 $1_L - 2_R$ 和 $2_L - 6_R$ ，表示把 $1 < 2 < 6$ 接成一条链。每条匹配边把两条链接起来，因此 m 条匹配边得到 $n - m$ 条链。

拆点边必须包含所有严格偏序关系。若输入只给出“较小指向较大”的 DAG 边，这些边通常只是偏序的生成关系，所以应先求传递闭包：只要 i 能到达 j ，就连接 $i_L - j_R$ 。

怎样实际取出反链

求出任意一组最大匹配 M 后，做一次交错 BFS：

1. 从所有未匹配的左点开始，并把它们标记为到达；
2. 当前在左点时，只沿非匹配边走到右点；
3. 当前在右点时，只沿它的匹配边走回左点；
4. 不能再扩展时停止，记到达的左右点集合分别为 Z_L, Z_R 。

方向始终是

$$\text{未匹配左点} \xrightarrow{\text{非匹配边}} R \xrightarrow{\text{匹配边}} L \xrightarrow{\text{非匹配边}} R \rightarrow \dots$$

不要从未匹配右点开始，也不要左点走匹配边。按上述方向得到的

$$C = (L \setminus Z_L) \cup Z_R$$

是拆点二分图的一组最小点覆盖。现在保留那些“左右副本都没有进入点覆盖”的原元素：

$$\begin{aligned} A &= \{i : i_L \notin C, i_R \notin C\} \\ &= \{i : i_L \in Z_L, i_R \notin Z_R\}. \end{aligned}$$

所以实现时只需判断 `reached_left[i] && !reached_right[i]`。

为什么这一定是最大反链：若 $i < j$ 且 i, j 都被选中，二分图中存在边 $i_L - j_R$ ，但它的两个端点都不在点覆盖 C 中，与 C 覆盖所有边矛盾，所以 A 是反链。又因为 $|C| = |M|$ ，至少有 $n - |C|$ 个元素的左右副本都不在 C 中；而匹配给出的 $n - |M|$ 条链说明任意反链至多从每条链取一个点。上下界相同，故 $|A| = n - |M|$ 。

例子：整除偏序

取元素 $\{1, 2, 3, 6\}$ ，规定 $x < y$ 当且仅当 x 严格整除 y 。拆点边为

$$1_L \rightarrow 2_R, \quad 1_L \rightarrow 3_R, \quad 1_L \rightarrow 6_R, \quad 2_L \rightarrow 6_R, \quad 3_L \rightarrow 6_R.$$

取最大匹配 $M = \{1_L - 2_R, 2_L - 6_R\}$ 。未匹配左点是 $3_L, 6_L$ ：从 3_L 沿非匹配边到 6_R ，再沿匹配边到 2_L ，之后无法继续。因此

$$Z_L = \{2_L, 3_L, 6_L\}, \quad Z_R = \{6_R\}.$$

逐个元素检查：

元素 i	$i_L \in Z_L$	$i_R \in Z_R$	是否进入反链
1	否	否	否
2	是	否	是
3	是	否	是
6	是	是	否

最终得到最大反链 $\{2, 3\}$ ，大小为 $4 - |M| = 2$ 。

```
// 使用上面的 HopcroftKarp。
vector<int> maximum_antichain(const vector<vector<int>>& dag) {
    int n = (int)dag.size();
    vector<vector<char>> less(n, vector<char>(n, false));

    // 求严格可达关系；输入必须是 DAG。
    for (int source = 0; source < n; ++source) {
        vector<char> visited(n, false);
        queue<int> que;
        visited[source] = true;
        que.push(source);
```

```

        while (!que.empty()) {
            int node = que.front();
            que.pop();
            for (int next : dag[node]) {
                if (visited[next]) continue;
                visited[next] = true;
                less[source][next] = true;
                que.push(next);
            }
        }
    }

    HopcroftKarp matching(n, n);
    for (int left = 0; left < n; ++left) {
        for (int right = 0; right < n; ++right) {
            if (less[left][right]) matching.add_edge(left, right);
        }
    }
    matching.max_matching();

    vector<char> reached_left(n, false), reached_right(n, false);
    queue<pair<int, bool>> que; // second=false 表示左点, true 表示右点
    for (int left = 0; left < n; ++left) {
        if (matching.match_left[left] == -1) {
            reached_left[left] = true;
            que.push({left, false});
        }
    }
    while (!que.empty()) {
        auto [node, is_right] = que.front();
        que.pop();
        if (!is_right) {
            // 左点只走不属于匹配的边。
            for (int right = 0; right < n; ++right) {
                if (!less[node][right] ||
                    matching.match_left[node] == right ||
                    reached_right[right]) {
                    continue;
                }
                reached_right[right] = true;
                que.push({right, true});
            }
        } else {
            // 右点只沿匹配边走回左点。
            int left = matching.match_right[node];
            if (left != -1 && !reached_left[left]) {
                reached_left[left] = true;
                que.push({left, false});
            }
        }
    }

    vector<int> antichain;

```

```

for (int element = 0; element < n; ++element) {
    if (reached_left[element] && !reached_right[element]) {
        antichain.push_back(element);
    }
}
return antichain;
}

```

方法二：原笔记的“链尾向链首跳”

上面的最大匹配还确定了一组最小链划分。若匹配边为 $u_L \rightarrow v_R$ ，就在原偏序中令 u 的后继为 v 、 v 的前驱为 u ：

```

successor[u] = matching.match_left[u];
predecessor[v] = matching.match_right[v];

```

满足 $\text{match_right}[\text{head}] == -1$ 的点是链首，满足 $\text{match_left}[\text{tail}] == -1$ 的点是链尾。你原来的构造可以严格写成：

1. 每条链先选择链尾，放入候选集合 H ；
2. 若当前代表 y 的右副本 y_R 被交错搜索到，即 $\text{reached_right}[y] == \text{true}$ ，就把 y 换成它在同一条链里的前驱 $\text{match_right}[y]$ ；
3. 继续向链首移动，直到当前代表的右副本没有被搜索到；每条链最终留下一个代表。

其中 $\text{reached_right}[y]$ 正是原笔记里“当前链尾与其他候选产生可比冲突，需要向链首跳”的统一标记。一次交错 BFS 会把连续产生的所有冲突同时传播出来，不必每次重新扫描 H 中的点对。

在前面函数已经求出 reached_right 后，也可以把最后构造答案的循环替换成下面这段：

```

vector<int> antichain;
for (int tail = 0; tail < n; ++tail) {
    if (matching.match_left[tail] != -1) continue; // 不是链尾
    int representative = tail;
    while (reached_right[representative]) {
        // 沿匹配边 representative_R -> predecessor_L 向链首跳一步。
        representative = matching.match_right[representative];
        assert(representative != -1);
    }
    antichain.push_back(representative);
}

```

它不会越过链首：如果某条链首 head 仍满足 $\text{reached_right}[\text{head}]$ ，就存在一条从未匹配左点到未匹配右点 head_R 的交错路，也就是增广路，这与当前匹配已经最大矛盾。

这个写法与方法一选出的是同一批点。链尾的左副本本来就是搜索起点；每向链首跳一步，走的正是匹配边 $\text{current}_R \rightarrow \text{predecessor}_L$ ，所以新代表的左副本仍被访问。停止时代表满足

```

reached_left[representative] && !reached_right[representative]

```

这恰好就是方法一的选点条件。在整除偏序的例子中，两条匹配链为 $1 < 2 < 6$ 和 3 ：先取链尾 $\{6, 3\}$ ，因为 6_R 被访问，所以把 6 向链首移动到 2 ，得到 $\{2, 3\}$ 。

传递闭包部分复杂度为 $O(n(n+m))$ ，闭包最多产生 n^2 条边；匹配复杂度为 $O(n^2\sqrt{n})$ 。若 n 较大且可达集合适合位运算，可换成 `bitset` 传递闭包。

二分图最大匹配的可行边与必须边

这里把平行边也视为不同的边：

- 可行边：至少属于一组最大匹配；
- 必须边：属于每一组最大匹配；
- 其余边不属于任何最大匹配。

先任取一组最大匹配 M ，仍把非匹配边定向为 $L \rightarrow R$ 、匹配边定向为 $R \rightarrow L$ 。记：

- `from_free_left`：从未匹配左点出发可达；
- `to_free_right`：能够到达某个未匹配右点，可在反图中从这些右点出发求出；
- `component`：上述有向图的强连通分量。

对非匹配边 (l, r) ，它可行当且仅当以下至少一项成立：两端同属一个 SCC、 l 可从自由左点到达、 r 可以到达自由右点。对匹配边，它本来就可行；若满足这三项中的任意一项，就能沿交错环或偶长交错路换掉，因此不是必须边，否则是必须边。存在完美匹配时没有自由点，判定恰好退化成 SCC 判据。

```
enum class MatchingEdgeType {
    impossible, // 不属于任何最大匹配
    feasible,   // 属于某些、但不属于所有最大匹配
    mandatory   // 属于每一组最大匹配
};

// 使用 02-tarjan.md 的 StronglyConnectedComponents。
// match_left、match_right 必须描述一组最大匹配。
vector<MatchingEdgeType> classify_maximum_matching_edges(
    int left_count, int right_count,
    const vector<pair<int, int>>& edges,
    const vector<int>& match_left,
    const vector<int>& match_right) {
    assert((int)match_left.size() == left_count);
    assert((int)match_right.size() == right_count);
    int node_count = left_count + right_count;
    for (int right : match_left) {
        assert(-1 <= right && right < right_count);
    }
    for (int left : match_right) {
        assert(-1 <= left && left < left_count);
    }

    vector<int> matched_edge(left_count, -1);
    for (int edge_id = 0; edge_id < (int)edges.size(); ++edge_id) {
        auto [left, right] = edges[edge_id];
        assert(0 <= left && left < left_count);
        assert(0 <= right && right < right_count);
        if (match_left[left] == right && match_right[right] == left &&
            matched_edge[left] == -1) {
            matched_edge[left] = edge_id;
        }
    }
    for (int left = 0; left < left_count; ++left) {
        if (match_left[left] != -1) {
            assert(match_right[match_left[left]] == left);
            assert(matched_edge[left] != -1);
        }
    }

    StronglyConnectedComponents scc(node_count);
    vector<vector<int>> directed(node_count), reversed(node_count);
```

```

auto add_directed_edge = [&](int from, int to) {
    directed[from].push_back(to);
    reversed[to].push_back(from);
    scc.add_edge(from, to);
};

for (int edge_id = 0; edge_id < (int)edges.size(); ++edge_id) {
    auto [left, right] = edges[edge_id];
    int right_node = left_count + right;
    if (matched_edge[left] == edge_id) {
        add_directed_edge(right_node, left);
    } else {
        add_directed_edge(left, right_node);
    }
}
scc.build();

auto reachable = [&](const vector<vector<int>>& graph,
                    const vector<int>& starts) {
    vector<char> visited(node_count, false);
    queue<int> que;
    for (int start : starts) {
        visited[start] = true;
        que.push(start);
    }
    while (!que.empty()) {
        int node = que.front();
        que.pop();
        for (int next : graph[node]) {
            if (visited[next]) continue;
            visited[next] = true;
            que.push(next);
        }
    }
    return visited;
};

vector<int> free_left, free_right;
for (int left = 0; left < left_count; ++left) {
    if (match_left[left] == -1) free_left.push_back(left);
}
for (int right = 0; right < right_count; ++right) {
    if (match_right[right] == -1) {
        free_right.push_back(left_count + right);
    }
}
vector<char> from_free_left = reachable(directed, free_left);
vector<char> to_free_right = reachable(reversed, free_right);

vector<MatchingEdgeType> result(
    edges.size(), MatchingEdgeType::impossible);
for (int edge_id = 0; edge_id < (int)edges.size(); ++edge_id) {
    auto [left, right] = edges[edge_id];

```



```

    int right_node = left_count + right;
    bool can_exchange =
        scc.component_of[left] == scc.component_of[right_node] ||
        from_free_left[left] || to_free_right[right_node];
    if (matched_edge[left] == edge_id) {
        result[edge_id] = can_exchange
            ? MatchingEdgeType::feasible
            : MatchingEdgeType::mandatory;
    } else if (can_exchange) {
        result[edge_id] = MatchingEdgeType::feasible;
    }
}
return result;
}

```

最大匹配求出后，建有向图、两次可达性和 SCC 总复杂度为 $O(V + E)$ 。不能对非完美匹配只使用“同 SCC”这一条，否则会漏掉通过自由点移动未匹配位置的可行边。

二分图最大权完美匹配

下面的 Hungarian 实现寻找“每个左点恰好匹配一个不同右点”的最大权匹配，要求 `left_count <= right_count`，并把权值矩阵中的每个位置都视为存在的边。不存在的边不能简单填 0：应填足够小的负数，并在结束后检查是否选到了它。

```

class MaximumWeightMatching {
    using i64 = long long;
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;

    int left_count;
    int right_count;
    vector<vector<i64>> weight;

public:
    MaximumWeightMatching(int left_size, int right_size)
        : left_count(left_size), right_count(right_size),
          weight(left_size, vector<i64>(right_size)) {
        assert(left_size <= right_size);
    }

    void set_weight(int left, int right, i64 value) {
        weight[left][right] = value;
    }

    pair<i64, vector<int>> solve() const {
        vector<i64> left_potential(left_count + 1);
        vector<i64> right_potential(right_count + 1);
        vector<int> matched_left(right_count + 1);
        vector<int> previous_right(right_count + 1);

        for (int left = 1; left <= left_count; ++left) {
            matched_left[0] = left;
            int right = 0;
            vector<i64> minimum(right_count + 1, inf);
            vector<bool> used(right_count + 1, false);

```

```

do {
    used[right] = true;
    int current_left = matched_left[right];
    i64 delta = inf;
    int next_right = 0;
    for (int candidate = 1; candidate <= right_count; ++candidate) {
        if (used[candidate]) continue;
        i64 reduced_cost =
            -weight[current_left - 1][candidate - 1] -
            left_potential[current_left] - right_potential[candidate];
        if (reduced_cost < minimum[candidate]) {
            minimum[candidate] = reduced_cost;
            previous_right[candidate] = right;
        }
        if (minimum[candidate] < delta) {
            delta = minimum[candidate];
            next_right = candidate;
        }
    }
    for (int candidate = 0; candidate <= right_count; ++candidate) {
        if (used[candidate]) {
            left_potential[matched_left[candidate]] += delta;
            right_potential[candidate] -= delta;
        } else {
            minimum[candidate] -= delta;
        }
    }
    right = next_right;
} while (matched_left[right] != 0);

do {
    int previous = previous_right[right];
    matched_left[right] = matched_left[previous];
    right = previous;
} while (right != 0);
}

vector<int> assignment(left_count, -1);
for (int right = 1; right <= right_count; ++right) {
    if (matched_left[right] != 0) {
        assignment[matched_left[right] - 1] = right - 1;
    }
}
i64 total_weight = 0;
for (int left = 0; left < left_count; ++left) {
    total_weight += weight[left][assignment[left]];
}
return {total_weight, assignment};
}
};

```

复杂度为 $O(n^2m)$ ，方阵时为 $O(n^3)$ 。权值取负以及势能运算必须能装入 long long。

一般图最大匹配 (Edmonds Blossom)

支持奇环, 点编号为 0-based。复杂度为 $O(V^3)$ 。

```
class GeneralMatching {
    int node_count;
    vector<vector<int>> graph;
    vector<int> parent;
    vector<int> base;
    vector<int> que;
    vector<bool> used;
    vector<bool> blossom;

    int lowest_common_ancestor(int lhs, int rhs) {
        vector<bool> visited(node_count, false);
        while (true) {
            lhs = base[lhs];
            visited[lhs] = true;
            if (match[lhs] == -1) break;
            lhs = parent[match[lhs]];
        }
        while (true) {
            rhs = base[rhs];
            if (visited[rhs]) return rhs;
            rhs = parent[match[rhs]];
        }
    }

    void mark_path(int node, int ancestor, int child) {
        while (base[node] != ancestor) {
            blossom[base[node]] = true;
            blossom[base[match[node]]] = true;
            parent[node] = child;
            child = match[node];
            node = parent[match[node]];
        }
    }

    bool find_augmenting_path(int root) {
        fill(used.begin(), used.end(), false);
        fill(parent.begin(), parent.end(), -1);
        iota(base.begin(), base.end(), 0);
        int que_begin = 0;
        int que_end = 0;
        que[que_end++] = root;
        used[root] = true;

        while (que_begin < que_end) {
            int node = que[que_begin++];
            for (int next : graph[node]) {
                if (base[node] == base[next] || match[node] == next) continue;
                if (next == root ||
                    (match[next] != -1 && parent[match[next]] != -1)) {
                    int ancestor = lowest_common_ancestor(node, next);
                    fill(blossom.begin(), blossom.end(), false);
                    mark_path(node, ancestor, next);
                }
            }
        }
    }
};
```

```

        mark_path(node, ancestor, next);
        mark_path(next, ancestor, node);
        for (int vertex = 0; vertex < node_count; ++vertex) {
            if (!blossom[base[vertex]]) continue;
            base[vertex] = ancestor;
            if (!used[vertex]) {
                used[vertex] = true;
                que[que_end++] = vertex;
            }
        }
    } else if (parent[next] == -1) {
        parent[next] = node;
        if (match[next] == -1) {
            int current = next;
            while (current != -1) {
                int previous = parent[current];
                int following = previous == -1 ? -1 : match[previous];
                match[current] = previous;
                if (previous != -1) match[previous] = current;
                current = following;
            }
            return true;
        }
        next = match[next];
        used[next] = true;
        que[que_end++] = next;
    }
}
}
return false;
}

public:
    vector<int> match;

    explicit GeneralMatching(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count),
          parent(initial_node_count), base(initial_node_count),
          que(initial_node_count), used(initial_node_count),
          blossom(initial_node_count), match(initial_node_count, -1) {}

    void add_edge(int lhs, int rhs) {
        assert(lhs != rhs);
        graph[lhs].push_back(rhs);
        graph[rhs].push_back(lhs);
    }

    int max_matching() {
        int matching_size = 0;
        for (int node = 0; node < node_count; ++node) {
            if (match[node] == -1 && find_augmenting_path(node)) {
                ++matching_size;
            }
        }
    }

```

```

    }
    return matching_size;
}
};

```

一般图匹配实现较长，比赛中尤其要测试三角形、五元环、两个奇环通过路径相连，以及含重边的情况。自环应在加入图前删除。

三元环与四元环计数

以下算法要求输入是无自环、无重边的简单无向图。按“度数优先、编号次优先”给点定全序，把每条边从较大的端点定向到较小的端点。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

i64 count_triangles(int node_count,
                    const vector<pair<int, int>>& edges) {
    vector<int> degree(node_count);
    for (auto [lhs, rhs] : edges) {
        ++degree[lhs];
        ++degree[rhs];
    }
    auto larger = [&](int lhs, int rhs) {
        return pair{degree[lhs], lhs} > pair{degree[rhs], rhs};
    };
    vector<vector<int>> outgoing(node_count);
    for (auto [lhs, rhs] : edges) {
        if (!larger(lhs, rhs)) swap(lhs, rhs);
        outgoing[lhs].push_back(rhs);
    }

    vector<bool> marked(node_count);
    i64 answer = 0;
    for (int largest = 0; largest < node_count; ++largest) {
        for (int next : outgoing[largest]) marked[next] = true;
        for (int middle : outgoing[largest]) {
            for (int smallest : outgoing[middle]) {
                answer += marked[smallest];
            }
        }
        for (int next : outgoing[largest]) marked[next] = false;
    }
    return answer;
}

i64 count_four_cycles(int node_count,
                      const vector<pair<int, int>>& edges) {
    vector<int> degree(node_count);
    vector<vector<int>> graph(node_count);

```

```

for (auto [lhs, rhs] : edges) {
    ++degree[lhs];
    ++degree[rhs];
    graph[lhs].push_back(rhs);
    graph[rhs].push_back(lhs);
}
auto order = [&](int node) {
    return pair{degree[node], node};
};
vector<vector<int>> outgoing(node_count);
for (auto [lhs, rhs] : edges) {
    if (order[lhs] < order[rhs]) swap(lhs, rhs);
    outgoing[lhs].push_back(rhs);
}

vector<int> path_count(node_count);
vector<int> touched;
i64 answer = 0;
for (int largest = 0; largest < node_count; ++largest) {
    for (int middle : outgoing[largest]) {
        for (int opposite : graph[middle]) {
            if (order[opposite] >= order[largest]) continue;
            if (path_count[opposite] == 0) touched.push_back(opposite);
            answer += path_count[opposite]++;
        }
    }
    for (int node : touched) path_count[node] = 0;
    touched.clear();
}
return answer;
}

```

三元环和四元环算法均利用低度定向把枚举量压到 $O(m\sqrt{m})$ 。四元环按顶点集计数；即使同一组四个点还有对角线，也仍只按其中实际存在的四边环计数。

图论常用技巧

最短路势能

若存在点势 $h(v)$ ，把边权改成 $w'(u, v) = w(u, v) + h(u) - h(v)$ ，则固定起终点的路径长度只相差常数，最短路不变。取 $h(v) = \text{dist}(s, v)$ 时，所有从可达点出发的边都有 $w' \geq 0$ ，这就是 Johnson 和势能费用流能使用 Dijkstra 的原因。

只给部分点设置随意势能可能产生负的约化边权；更新势能时只更新本轮 Dijkstra 可达的点。

最短路 DAG

求出源点距离后，保留满足 $\text{dist}(u) + w(u, v) = \text{dist}(v)$ 的边，得到所有最短路组成的子图。正权图中它按距离严格递增，是 DAG，可在上面计数、求必经点。存在零权边时未必是 DAG，需要先缩零权强连通分量。

MST 换边

- 加入非树边 (u, v, w) 后形成唯一环；删除环上最大权树边可得到包含该边的最优生成树。

- 强制删除树边后形成两个连通块；跨越割的最小非树边是最佳替代边。
- 判断严格次小生成树时，需要区分路径最大值与“严格小于新边权的最大值”，否则相同权值只会得到另一棵同权 MST。

树上路径最大边可用倍增或树链剖分维护。

补图 BFS

在稠密图的补图上 BFS 时，不要显式建立 $O(n^2)$ 条边。维护尚未访问点的集合；处理 u 时先标记原图邻居，再从未访问集合中删除所有未被标记的点并加入队列。每个点只被删除一次，原图边只用于标记，总复杂度可做到 $O(n + m)$ 或带集合对数因子的版本。

函数图

每个点出度恰为 1 的图由“有向环 + 指向环的内向树”组成。常见处理顺序：

1. 用入度拓扑删除非环点，剩余点都在环上；
2. 分别给每个环编号并记录环长；
3. 在反图上从环点向外搜索，得到所属环、到环距离和树上祖先。

这样可以统一处理第 k 个后继、两点何时相遇以及路径长度问题。

欧拉路检查表

- 无向图：忽略零度点后连通；欧拉回路要求奇度点为 0，欧拉路径要求奇度点为 2。
- 有向图：忽略零度点后弱连通；回路要求所有点入度等于出度，路径允许起点出度多 1、终点入度多 1。
- Hierholzer 中每条边只删除一次。无向边必须用同一个边编号同时标记两个方向。

离线删除

只有删边而没有加边的问题，可以倒序处理：先加入最终仍存在的边，再把删除操作逆序变成加边，用并查集维护连通性。若有时间区间上的加删混合，可使用线段树分治加可撤销并查集。

Kruskal 重构树

按边权从小到大做 Kruskal。每次合并两个连通块时新建一个虚点，点权等于当前边权，并把两个连通块的根接到虚点下。树上叶子是原图顶点，父亲点权单调不降。

对不同顶点 u, v ，重构树中 $\text{lca}(u, v)$ 的点权等于使二者首次连通的最小边权，也等于原图中所有 $u \rightarrow v$ 路径的“最大边权最小值”。图不连通时得到森林。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;

class KruskalReconstructionTree {
public:
    struct Edge {
        int from;
        int to;
        i64 weight;
    };
};
```

```

private:
    int original_size;
    int node_count;
    int logarithm;
    vector<int> dsu_parent;
    vector<vector<int>> children;
    vector<i64> node_weight;
    vector<int> depth;
    vector<int> component;
    vector<vector<int>> ancestor;

    int find_set(int node) {
        int root = node;
        while (dsu_parent[root] != root) root = dsu_parent[root];
        while (dsu_parent[node] != node) {
            int next = dsu_parent[node];
            dsu_parent[node] = root;
            node = next;
        }
        return root;
    }

public:
    KruskalReconstructionTree(int vertex_count, vector<Edge> edges)
        : original_size(vertex_count), node_count(vertex_count), logarithm(1),
          dsu_parent(max(1, 2 * vertex_count)),
          children(max(1, 2 * vertex_count)),
          node_weight(max(1, 2 * vertex_count),
                      numeric_limits<i64>::lowest()) {
        assert(vertex_count >= 0);
        iota(dsu_parent.begin(), dsu_parent.end(), 0);
        sort(edges.begin(), edges.end(), [](const Edge& left, const Edge& right) {
            return left.weight < right.weight;
        });

        for (const Edge& edge : edges) {
            assert(0 <= edge.from && edge.from < original_size);
            assert(0 <= edge.to && edge.to < original_size);
            int left_root = find_set(edge.from);
            int right_root = find_set(edge.to);
            if (left_root == right_root) continue;

            int merged = node_count++;
            node_weight[merged] = edge.weight;
            children[merged] = {left_root, right_root};
            dsu_parent[left_root] = merged;
            dsu_parent[right_root] = merged;
            dsu_parent[merged] = merged;
        }

        children.resize(node_count);
        node_weight.resize(node_count);
        while ((1 << logarithm) <= max(1, node_count)) ++logarithm;
    }

```



```

depth.assign(node_count, 0);
component.assign(node_count, -1);
ancestor.assign(logarithm, vector<int>(node_count, -1));

for (int node = 0; node < node_count; ++node) {
    if (find_set(node) != node) continue;
    component[node] = node;
    vector<int> stack{node};
    while (!stack.empty()) {
        int current = stack.back();
        stack.pop_back();
        for (int child : children[current]) {
            depth[child] = depth[current] + 1;
            component[child] = node;
            ancestor[0][child] = current;
            stack.push_back(child);
        }
    }
}

for (int level = 1; level < logarithm; ++level) {
    for (int node = 0; node < node_count; ++node) {
        int middle = ancestor[level - 1][node];
        if (middle != -1) {
            ancestor[level][node] = ancestor[level - 1][middle];
        }
    }
}

bool connected(int left, int right) const {
    assert(0 <= left && left < original_size);
    assert(0 <= right && right < original_size);
    return component[left] == component[right];
}

int lca_node(int left, int right) const {
    assert(connected(left, right));
    if (depth[left] < depth[right]) swap(left, right);
    int difference = depth[left] - depth[right];
    for (int level = 0; level < logarithm; ++level) {
        if (difference >> level & 1) left = ancestor[level][left];
    }
    if (left == right) return left;
    for (int level = logarithm - 1; level >= 0; --level) {
        if (ancestor[level][left] != ancestor[level][right]) {
            left = ancestor[level][left];
            right = ancestor[level][right];
        }
    }
    return ancestor[0][left];
}

i64 minimum_bottleneck(int left, int right) const {

```

```

    assert(left != right && connected(left, right));
    return node_weight[lca_node(left, right)];
}

int size() const { return node_count; }
const vector<vector<int>>& forest() const { return children; }
const vector<i64>& weights() const { return node_weight; }
};

```

排序复杂度 $O(m \log m)$ ，建树后的倍增预处理为 $O(n \log n)$ ，单次查询 $O(\log n)$ 。若改用 Euler 序 RMQ，LCA 可以做到 $O(1)$ 。

常见离线题还会在重构树上做子树统计：给定边权上限 w ，从叶子向上倍增到点权不超过 w 的最高祖先，该祖先的叶子集合正是此时的连通块。

Euler 序 RMQ 求 LCA

DFS 每次进入结点以及从儿子返回时都把当前结点加入 Euler 序。两点第一次出现位置之间，深度最小的结点就是 LCA；不要把只记录首次进入的普通 DFS 序误当成这条长度 $2n - 1$ 的 Euler 序。下面用迭代 DFS 避免深树递归爆栈，并用 Sparse Table 做 RMQ。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class EulerTourLca {
    vector<int> first;
    vector<int> depth;
    vector<int> component;
    vector<int> euler;
    vector<int> logarithm;
    vector<vector<int>> sparse_table;

public:
    explicit EulerTourLca(const vector<vector<int>>& graph,
                          const vector<int>& preferred_roots = {}) {
        int n = (int)graph.size();
        first.assign(n, -1);
        depth.assign(n, 0);
        component.assign(n, -1);

        vector<int> roots = preferred_roots;
        for (int node = 0; node < n; ++node) roots.push_back(node);
        vector<int> parent(n, -1);

        struct Frame {
            int node;
            int next_edge;
        };

        for (int root : roots) {
            assert(0 <= root && root < n);
            if (component[root] != -1) continue;

```

```

component[root] = root;
first[root] = (int)euler.size();
euler.push_back(root);
vector<Frame> stack{{root, 0}};

while (!stack.empty()) {
    Frame& frame = stack.back();
    int node = frame.node;
    if (frame.next_edge == (int)graph[node].size()) {
        stack.pop_back();
        if (!stack.empty()) euler.push_back(stack.back().node);
        continue;
    }
    int next = graph[node][frame.next_edge++];
    if (next == parent[node] || component[next] != -1) continue;
    parent[next] = node;
    depth[next] = depth[node] + 1;
    component[next] = root;
    first[next] = (int)euler.size();
    euler.push_back(next);
    stack.push_back({next, 0});
}

}

logarithm.assign(euler.size() + 1, 0);
for (int length = 2; length <= (int)euler.size(); ++length) {
    logarithm[length] = logarithm[length / 2] + 1;
}
int levels = euler.empty() ? 0 : logarithm[euler.size()] + 1;
sparse_table.assign(levels, vector<int>(euler.size()));
if (!euler.empty()) sparse_table[0] = euler;
for (int level = 1; level < levels; ++level) {
    int length = 1 << level;
    for (int begin = 0; begin + length <= (int)euler.size(); ++begin) {
        int left = sparse_table[level - 1][begin];
        int right = sparse_table[level - 1][begin + length / 2];
        sparse_table[level][begin] =
            depth[left] <= depth[right] ? left : right;
    }
}

}

int lca(int left, int right) const {
    assert(0 <= left && left < (int)first.size());
    assert(0 <= right && right < (int)first.size());
    assert(component[left] == component[right]);
    int begin = first[left];
    int end = first[right];
    if (begin > end) swap(begin, end);
    int level = logarithm[end - begin + 1];
    int lhs = sparse_table[level][begin];
    int rhs = sparse_table[level][end - (1 << level) + 1];
    return depth[lhs] <= depth[rhs] ? lhs : rhs;
}

```

```

    }

    int distance(int left, int right) const {
        int ancestor = lca(left, right);
        return depth[left] + depth[right] - 2 * depth[ancestor];
    }
};

```

预处理时间和空间均为 $O(n \log n)$ ，查询 $O(1)$ 。输入可以是森林，但只能查询同一连通分量内的两点。

Hierholzer 欧拉路

Hierholzer 的关键是为每个点维护一个只增指针。每条邻接弧只会被指针越过一次，因此有向图为 $O(V + E)$ ，无向图虽然每条边存两条弧，仍是 $O(V + E)$ ，不能在每一步从 `vector` 中线性删除边。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class EulerTrail {
public:
    struct Result {
        vector<int> vertices;
        vector<int> edge_ids;
    };

private:
    struct Arc {
        int to;
        int edge_id;
    };

    int vertex_count;
    bool directed;
    int edge_count = 0;
    vector<vector<Arc>> graph;
    vector<int> indegree;
    vector<int> outdegree;

public:
    EulerTrail(int initial_vertex_count, bool is_directed)
        : vertex_count(initial_vertex_count), directed(is_directed),
          graph(initial_vertex_count), indegree(initial_vertex_count),
          outdegree(initial_vertex_count) {
        assert(initial_vertex_count >= 0);
    }

    int add_edge(int from, int to) {
        assert(0 <= from && from < vertex_count);
        assert(0 <= to && to < vertex_count);
        int id = edge_count++;
        graph[from].push_back({to, id});
    }
};

```

```

    ++outdegree[from];
    ++indegree[to];
    if (!directed) {
        graph[to].push_back({from, id});
        ++outdegree[to];
        ++indegree[from];
    }
    return id;
}

optional<Result> find_trail() const {
    if (vertex_count == 0) {
        if (edge_count == 0) return Result{};
        return nullopt;
    }

    int start = -1;
    if (directed) {
        int start_candidates = 0;
        int end_candidates = 0;
        for (int node = 0; node < vertex_count; ++node) {
            int difference = outdegree[node] - indegree[node];
            if (difference == 1) {
                start = node;
                ++start_candidates;
            } else if (difference == -1) {
                ++end_candidates;
            } else if (difference != 0) {
                return nullopt;
            }
        }
        if (start_candidates != end_candidates || start_candidates > 1) {
            return nullopt;
        }
        if (start == -1) {
            for (int node = 0; node < vertex_count; ++node) {
                if (outdegree[node] > 0) {
                    start = node;
                    break;
                }
            }
        }
    } else {
        int odd_count = 0;
        for (int node = 0; node < vertex_count; ++node) {
            if (outdegree[node] & 1) {
                start = node;
                ++odd_count;
            }
        }
        if (odd_count != 0 && odd_count != 2) return nullopt;
        if (start == -1) {
            for (int node = 0; node < vertex_count; ++node) {

```

```

        if (outdegree[node] > 0) {
            start = node;
            break;
        }
    }
}

if (start == -1) return Result{{0}, {}};

vector<int> next_arc(vertex_count);
vector<bool> used(edge_count);
vector<int> vertex_stack{start};
vector<int> incoming_edge{-1};
Result reversed;

while (!vertex_stack.empty()) {
    int node = vertex_stack.back();
    while (next_arc[node] < (int)graph[node].size() &&
        used[graph[node][next_arc[node]].edge_id]) {
        ++next_arc[node];
    }
    if (next_arc[node] == (int)graph[node].size()) {
        reversed.vertices.push_back(node);
        int edge_id = incoming_edge.back();
        if (edge_id != -1) reversed.edge_ids.push_back(edge_id);
        vertex_stack.pop_back();
        incoming_edge.pop_back();
        continue;
    }

    Arc arc = graph[node][next_arc[node]++];
    if (used[arc.edge_id]) continue;
    used[arc.edge_id] = true;
    vertex_stack.push_back(arc.to);
    incoming_edge.push_back(arc.edge_id);
}

if ((int)reversed.edge_ids.size() != edge_count) return nullopt;
reverse(reversed.vertices.begin(), reversed.vertices.end());
reverse(reversed.edge_ids.begin(), reversed.edge_ids.end());
return reversed;
}
};

```

度数条件只解决局部平衡；非零度顶点若不连通，最终使用的边数会小于 `edge_count`，模板会返回无解。返回的 `edge_ids[i]` 对应 `vertices[i] -> vertices[i+1]`，因此平行边和自环也能区分。

2-SAT

`add_or(x, x_value, y, y_value)` 加入子句 $(x == x_value) \vee (y == y_value)$ 。每个文字是一个点，子句 $a \vee b$ 加两条蕴含边 $\neg a \rightarrow b$ 、 $\neg b \rightarrow a$ 。单独加入蕴含 $a \Rightarrow b$ 时也必须同时加入逆否命题 $\neg b \Rightarrow \neg a$ ；构造赋值

应按当前 SCC 实现的编号方向，不能套用别的模板的大小关系。

```
#include <bits/stdc++.h>
using namespace std;

struct TwoSat {
    int variable_count, timer = 0, component_count = 0;
    vector<vector<int>> graph;
    vector<int> dfn, low, component, stack;
    vector<bool> in_stack;

    explicit TwoSat(int n) : variable_count(n), graph(2 * n), dfn(2 * n),
        low(2 * n), component(2 * n, -1), in_stack(2 * n) {}

    int literal(int variable, bool value) const {
        return 2 * variable + value;
    }

    void add_or(int x, bool x_value, int y, bool y_value) {
        int a = literal(x, x_value);
        int b = literal(y, y_value);
        graph[a ^ 1].push_back(b);
        graph[b ^ 1].push_back(a);
    }

    void force(int x, bool value) {
        add_or(x, value, x, value);
    }

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        stack.push_back(u);
        in_stack[u] = true;
        for (int v : graph[u]) {
            if (dfn[v] == 0) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_stack[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] != dfn[u]) return;
        while (true) {
            int v = stack.back();
            stack.pop_back();
            in_stack[v] = false;
            component[v] = component_count;
            if (v == u) break;
        }
        ++component_count;
    }

    optional<vector<bool>> solve() {
        for (int u = 0; u < 2 * variable_count; ++u) {
```

```

        if (dfn[u] == 0) dfs(u);
    }
    vector<bool> answer(variable_count);
    for (int x = 0; x < variable_count; ++x) {
        if (component[2 * x] == component[2 * x + 1]) return nullopt;
        // Tarjan 弹栈顺序是反拓扑序，编号较小的 SCC 先取真。
        answer[x] = component[2 * x + 1] < component[2 * x];
    }
    return answer;
}
};

```

时间与空间复杂度都是 $O(n + m)$ 。solve() 只应调用一次；若继续加子句，应重新建对象。递归深度最坏为 $O(n)$ ，极大稀疏图需要留意栈空间。

Capacity scaling 最小费用循环流

普通逐次最短路每次增广的流量可能只有 1，复杂度会依赖总流量。Capacity scaling 把容量从高位到低位加入：进入下一位时，先把当前容量和流量都乘 2，再为这一位为 1 的边增加一个单位容量。

维护势能 p ，约化费用定义为

$$c_p(u, v) = p(u) + c(u, v) - p(v).$$

若当前所有残量边的约化费用非负，则残量图没有负圈，当前循环流最优。给原边 $u \rightarrow v$ 新增一个单位容量时，只有这条新残量边可能破坏条件：

- 若它原本已有正残量，只增加容量，不会出现一种新的环；
- 否则，在旧残量图中从 v 跑 Dijkstra。若“新边 $u \rightarrow v$ + 最短路 $v \rightsquigarrow u$ ”费用为负，就沿该负圈增广一个单位；否则保留新边的残量容量；
- 用最短路距离更新势能后，所有现存残量边的约化费用重新非负。

每条边的每个二进制位至多触发一次 Dijkstra，共 $O(m \log U)$ 次；使用二叉堆时复杂度为 $O(m(n + m) \log n \log U)$ ，通常写作 $O(m^2 \log n \log U)$ 。这里 U 是最大容量。

模板

这是最小费用循环流：没有源汇，每个点流量守恒，允许算法主动增广负圈。add_edge 返回输入边编号，答案中的 edge_flow[id] 与之对应。容量和费用用 long long 保存，距离、势能和总费用临时使用 __int128。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class CapacityScalingMinCostCirculation {
    using i64 = long long;
    using i128 = __int128_t;
    static constexpr i128 infinity = (i128(1) << 120);

    struct Arc {
        int from, to;
        i64 original_capacity;
        i64 residual_capacity;
    };

```



```

        i64 cost;
    };

    int node_count;
    bool solved = false;
    vector<Arc> arcs;
    vector<vector<int>> graph;
    vector<int> original_arcs;
    vector<i128> potential, distance;
    vector<int> previous_arc;

    i128 reduced_cost(int arc_id) const {
        const Arc& arc = arcs[arc_id];
        return potential[arc.from] + arc.cost - potential[arc.to];
    }

    void dijkstra(const vector<i128>& initial_distance) {
        using State = pair<i128, int>;
        priority_queue<State, vector<State>, greater<State>> que;
        distance = initial_distance;
        previous_arc.assign(node_count, -1);
        for (int node = 0; node < node_count; ++node) {
            if (distance[node] != infinity) {
                que.push({distance[node], node});
            }
        }

        while (!que.empty()) {
            auto [current_distance, node] = que.top();
            que.pop();
            if (current_distance != distance[node]) continue;
            for (int arc_id : graph[node]) {
                const Arc& arc = arcs[arc_id];
                if (arc.residual_capacity == 0) continue;
                i128 length = reduced_cost(arc_id);
                assert(length >= 0);
                i128 next_distance = current_distance + length;
                if (next_distance < distance[arc.to]) {
                    distance[arc.to] = next_distance;
                    previous_arc[arc.to] = arc_id;
                    que.push({next_distance, arc.to});
                }
            }
        }
    }

    // 势能只差一个整体常数：重新取虚拟源到各点的最短路，防止其不断漂移。
    void normalize_potential() {
        i128 maximum = *max_element(potential.begin(), potential.end());
        vector<i128> initial_distance(node_count);
        for (int node = 0; node < node_count; ++node) {
            initial_distance[node] = maximum - potential[node];
        }
    }
}

```

```

    dijkstra(initial_distance);
    for (int node = 0; node < node_count; ++node) {
        potential[node] += distance[node] - maximum;
    }
}

void add_one_capacity(int arc_id) {
    Arc& added_arc = arcs[arc_id];
    if (added_arc.residual_capacity > 0) {
        ++added_arc.residual_capacity;
        return;
    }

    int from = added_arc.from;
    int to = added_arc.to;
    i128 added_length = reduced_cost(arc_id);
    vector<i128> initial_distance(node_count, infinity);
    initial_distance[to] = 0;
    dijkstra(initial_distance);

    if (distance[from] != infinity &&
        distance[from] + added_length < 0) {
        // 使用新边, 并沿最短路 to -> from 增广一个单位。
        ++arcs[arc_id ^ 1].residual_capacity;
        for (int node = from; node != to; ) {
            int path_arc = previous_arc[node];
            assert(path_arc != -1);
            --arcs[path_arc].residual_capacity;
            ++arcs[path_arc ^ 1].residual_capacity;
            node = arcs[path_arc].from;
        }
    } else {
        ++arcs[arc_id].residual_capacity;
    }

    i128 maximum_distance = 0;
    for (i128 value : distance) {
        if (value != infinity) maximum_distance = max(maximum_distance, value);
    }
    i128 unreachable_distance =
        maximum_distance + max<i128>(0, -added_length);
    for (int node = 0; node < node_count; ++node) {
        potential[node] += distance[node] == infinity
            ? unreachable_distance : distance[node];
    }
    normalize_potential();
}

public:
    struct Result {
        i128 cost;
        vector<i64> edge_flow;
    };
};

```

```

explicit CapacityScalingMinCostCirculation(int initial_node_count)
    : node_count(initial_node_count), graph(initial_node_count),
      potential(initial_node_count) {
    assert(node_count > 0);
}

int add_edge(int from, int to, i64 capacity, i64 cost) {
    assert(!solved);
    assert(0 <= from && from < node_count);
    assert(0 <= to && to < node_count);
    assert(capacity >= 0);
    assert(cost != numeric_limits<i64>::min());
    int arc_id = (int)arcs.size();
    arcs.push_back({from, to, capacity, 0, cost});
    arcs.push_back({to, from, 0, 0, -cost});
    graph[from].push_back(arc_id);
    graph[to].push_back(arc_id ^ 1);
    original_arcs.push_back(arc_id);
    return (int)original_arcs.size() - 1;
}

Result solve() {
    assert(!solved);
    solved = true;
    int highest_bit = -1;
    for (int arc_id : original_arcs) {
        unsigned long long capacity = arcs[arc_id].original_capacity;
        if (capacity > 0) {
            highest_bit = max(highest_bit,
                              63 - __builtin_clzll(capacity));
        }
    }

    for (int bit = highest_bit; bit >= 0; --bit) {
        for (Arc& arc : arcs) arc.residual_capacity *= 2;
        for (int arc_id : original_arcs) {
            unsigned long long capacity = arcs[arc_id].original_capacity;
            if ((capacity >> bit) & 1ULL) add_one_capacity(arc_id);
        }
    }

    Result result{0, vector<i64>(original_arcs.size())};
    for (int id = 0; id < (int)original_arcs.size(); ++id) {
        int arc_id = original_arcs[id];
        i64 flow = arcs[arc_id ^ 1].residual_capacity;
        result.edge_flow[id] = flow;
        result.cost += (i128)flow * arcs[arc_id].cost;
    }
    return result;
}
};

```

`solve()` 只能调用一次。模板允许平行边、自环和负费用边；`cost == LLONG_MIN` 被排除，因为建立反向边

时 `-cost` 无法放进 `long long`。应另写 `__int128` 输出函数打印总费用。

从循环流得到最小费用最大流

文章中的做法是加辅助边 `sink -> source`，容量取流量上界，费用取一个绝对值足够大的负数，然后求最小费用循环流；辅助边流量就是最大流量。不过“足够大”必须有证明。若所有原边满足 $|c| \leq C$ ，则简单增广路至多有 $n - 1$ 条边，可取

$$K = (n - 1)C + 1,$$

并令辅助边费用为 $-K$ 。还必须保证 K 、辅助容量及答案均不会溢出；不能直接写 `-1e18`。

更稳妥的赛场方案是分两步：先忽略费用求最大流值 F ，再加入 `sink -> source`、下界和上界都为 F 、费用为 0 的边，把问题变成固定流量的最小费用可行循环流。这样不依赖人造负费用常数。

有负费用边、负圈时的上下界做法

逐次最短路只从源点增广，可能漏掉与源汇无关的负圈。对有限上下界边

$$u \rightarrow v, \quad l \leq f \leq r, \quad \text{费用 } c,$$

可以先把每条边放在局部费用最小的端点，从而把剩余调整边全部变成非负费用：

- 若 $c \geq 0$ ，预先令 $f = l$ ，加入 `u -> v`、容量 $r - l$ 、费用 c 的调整边；
- 若 $c < 0$ ，预先令 $f = r$ ，加入 `v -> u`、容量 $r - l$ 、费用 $-c$ 的调整边。调整边上的流量表示从满流中撤回多少；
- 预流为 f_0 时，更新 `balance[u] -= f0`、`balance[v] += f0`，基础费用加上 $f_0 c$ 。

此时所有调整边费用非负。对 `balance[v] > 0` 加 `super_source -> v`，对 `balance[v] < 0` 加 `v -> super_sink`，用最小费用最大流把超级源的所有需求送满：送不满则无可行解，送满时就是全局最小费用解。恢复原边流量时：

- $c \geq 0$: $f = l + x$;
- $c < 0$: $f = r - x$;

其中 x 是对应调整边的流量。最终费用也可写成“基础费用 + 调整费用”。这个变换已经把原问题中可能出现的负圈吸收到“负费用边默认取上界”里，因此不能只做一次普通可行流后就停下。

要求固定 $s \rightarrow t$ 流量为 F 时，再加入 `t -> s`、下界和上界均为 F 、费用为 0 的边，然后做同一套变换即可。求最小费用最大流时，先求最大流值 F ，再固定 F ，最不容易踩边界。

参考：ouuan 的推导与实现，MIT 6.854 Min-Cost Flow 讲义。